



FOR SOFTWARE ENGINEERS: JUDGE THE DIFF, NOT THE EXPLANATION

ChatGPT and Codex

The review that passed, and was wrong

Three Java reviews in seconds. Two were right. The wrong one was the most convincing of the three.

That is the whole problem with this way of working, and it is why this is not a sheet about prompts.

What is here instead: which of the three modes you should actually be in, the permissions dialog most people click through, the one file that raises the quality of everything, and a worked example of a review that sounded authoritative and was wrong in the most expensive direction available.

Every product claim on this sheet was read from the vendor documentation in September 2026 and is stated as at that date. This is product surface that moves in months, so the date is the point of the page rather than decoration.

In scope

- Three genuinely different tools in one window, and how to tell which you need
- What the sandbox defaults actually are, which are better than people assume
- The approval policy that can switch the safety off without looking like it
- AGENTS.md, and why it is the highest leverage file in the repository
- A real wrong review, why it was wrong, and what the correct fix was

Out of scope

- Prompt phrasing. The failures here are not caused by wording
- Benchmark comparisons between assistants
- Any claim that one tool is better than another at software
- Anything about how this channel is produced, which is not the subject

If you read one section, read the wrong review. It is the one that changes how you treat the output rather than how you write the input.

01 The sandbox defaults, September 2026

Network access	Off by default
Workspace mode	Can write the working directory
.git, .agents, .codex	Protected, so history cannot be rewritten by accident
Folder not under version control	Read only recommended, because no history means no undo
Approval policy	Ask on request, never, or granular by category
Full access	Removes the boundary automatic review depends on

Better than most people assume, and they are defaults rather than guarantees. The approval policy above them can remove the boundary the automatic review depends on.

03 Three modes, one window

The thing almost everyone gets wrong on day one is that the desktop app holds three genuinely different tools.

Chat is for questions and search. Work is the agent for longer jobs that end in a deliverable: documents, spreadsheets, reports. Codex is for software, with local files, repositories, terminals and developer tools.

If you are asking Chat to refactor your service layer, you are using the wrong one, and no amount of prompt craft fixes that.

The desktop application presents Chat, Work and Codex as distinct modes, with Codex described as the workspace for developing software with local files, repositories and terminals.

Help centre, ChatGPT Work and Codex, September 2026

Mode selection is a product distinction rather than a prompting one, so it cannot be compensated for by better instructions.

First principles, not vendor documentation

04 The dialog most people click through

Folder access, terminal and command access, browser use and developer tools, each with deny and allow. Read the line underneath those toggles, because it is the best advice in the product: only allow what the task needs.

An agent with terminal access can run anything you can run. An agent with browser access is signed in as you. There is no undo button on a shell command.

The defaults are better than people assume. At the time of writing, in September 2026, it runs with network access off by default, and in workspace mode it can write your working directory while .git, .agents and .codex stay protected, so it cannot rewrite your history by accident.

For a folder not under version control at all, read only is the recommended default, and the reason is worth repeating: no git history means no undo.

The agent runs with network access disabled by default.

Agent approvals and security documentation, September 2026

In workspace write mode the working directory is writable while .git, .agents and .codex remain protected.

Agent approvals and security documentation, September 2026

Read only is the recommended default for a folder that is not under version control.

Agent approvals and security documentation, September 2026

05 The caveat an advertisement would leave out

Above the sandbox sits the approval policy: ask on request, never, or a granular policy set category by category.

Full access removes the sandbox boundary that automatic review depends on. Set the policy to never, or run with full access, and a sensitive action may never produce an approval request for you to see. Organisations can block those options outright for exactly that reason.

The safety here is real, and it is real BECAUSE OF THE BOUNDARY. Turn the boundary off and you have turned the safety off with it.

Approval policies include asking on request, never asking, and a granular per-category policy.

Agent approvals and security documentation, September 2026

Running with full access removes the sandbox boundary that the automatic review of sensitive actions depends on.

Agent approvals and security documentation, September 2026

06 AGENTS.md is the highest leverage file in your repository

If you take one practical thing from this sheet, take this one. It loads into context automatically, every session, with nobody remembering to paste anything.

Three headings carry most of the value. COMMANDS: the build, the test, the one that needs docker compose up first. CONSTRAINTS: never call a payment provider inside a transaction, every consumer idempotent on the payment id. DONE MEANS: tests pass, and the diff touches only the files named above.

Every engineer on your team is currently re-explaining the same project context by hand, several times a day, and the explanations disagree with each other. Write it down once.

AGENTS.md is read automatically as project context at the start of a session.

Codex manual, retrieved in full and grepped, September 2026

A written definition of done gives the agent a check it can run against, where an unstated one gives it nothing.

Engineering practice, not vendor documentation



07 The review that was wrong

The prompt was ordinary: review this service for correctness under failure, be specific. The answer came back saying the method was correctly transactional, and suggesting the transaction timeout be raised so the payment call had room to complete.

It sounds authoritative. It is wrong, and it is wrong in the most expensive direction available.

A remote call inside a transaction holds a database connection open for the length of somebody else's network, and when that call fails your database rolls back while the customer's card stays charged. Raising the timeout makes it worse.

The fix is to SHORTEN the transaction, not lengthen it. Write the order and an outbox row, commit, then call the provider outside the transaction using their own idempotency key so a retry cannot charge twice.

A local database transaction cannot roll back an effect that occurred in a remote system, so a rollback after a successful charge leaves the two in disagreement.

First principles, not vendor documentation

Holding a transaction open across a remote call binds a connection for the duration of a third party network, which raising the timeout extends rather than fixes.

Run against real Java on the machine that built this sheet

08 What to take from it

Invented interfaces. Confident wrong assumptions. Abstractions nobody asked for. Code that compiles perfectly and breaks a business rule.

None of them look like errors. They look like good work, and that is exactly what makes them expensive.

Judge the diff and the test results, never how convincing the explanation sounded. That sentence is the sheet.

The failure modes that cost the most are the ones that are indistinguishable from competence at a glance.

First principles, not vendor documentation

Before you accept a review

1. Am I in the right mode for this task, or asking Chat to do engineering?
2. Does this task actually need terminal access, or did I allow it out of habit?
3. Does it need the browser, given it would be signed in as me?
4. Is this folder under version control, and if not why is it writable?
5. Is the approval policy still asking, or did somebody set it to never?
6. Does the repository have an AGENTS.md, and is it current?
7. Does that file say what DONE means, in a form something can check?
8. Am I judging the diff, or the confidence of the explanation?
9. Is there a test that would settle this claim faster than reading it?
10. If this review is wrong, which direction is the expensive one?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which review it got wrong for you.

ChatGPT and Codex are products of OpenAI. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by OpenAI, and no OpenAI artwork is reproduced here. Product names appear only to identify the software being discussed. Java is a trademark of Oracle and/or its affiliates. Other names are the property of their respective owners.

Technical claims verified 2026-09-14 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.